

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI GenAI 기반 기술



CONTENTS

목차

- 01 AI·프로그래밍 패러다임 변천사와 모델 생태계
- 02 LLM 텍스트 생성 원리와 생성 파라미터
- 03 나만의 AI 어시스턴트 만들기 (Gemini Gems)
- 04 프롬프트의 단계적 개선
- 05 AI 코딩 에이전트 설치와 구조 (Claude Code)
- 06 멀티 LLM 통합과 첫 API 서비스 (LiteLLM·FastAPI)

과정 전체 지도



GenAI 기반 기술

AI·프로그래밍 패러다임 변천사와 모델 생태계

변천사 / 모델 생태계 / 비용 구조

학습 목표

- 분석형 AI, 생성형 AI, Agentic AI의 변천을 무엇이 로직을 만드는가와 누가 실행을 지휘하는가로 구별해 설명할 수 있습니다.
- 무료 웹, 구독, API 종량 중 상황에 맞는 접근 방식을 골라 그 이유를 말할 수 있습니다.

▶ 변천사와 모델 생태계

텍스트 생성 원리와 파라미터
나만의 AI 어시스턴트
프롬프트의 단계적 개선
AI 코딩 에이전트 설치와 구조
멀티 LLM 통합과 첫 API 서비스

두 개의 채용 공고

세상이 뽑는 사람이 바뀌었습니다. 무엇이 바뀌어서 이렇게 되었는지부터 봅니다.

데이터 분석가

"SQL·통계 기반 데이터 분석"

SQL을 활용해 복잡한 데이터세트를 다룬 실무 경험.
A/B 테스트 설계, 가설 검정과 회귀 등
통계 기법으로 문제를 해결한 경험.

AI Agent 개발자

"LLM 기반 에이전트 설계·운영"

LLM 기반 AI Agent 시스템 설계와 개발.
RAG 기반 문서 검색·질의응답 시스템 개발.
LangGraph, RAG, Vector DB.

분석형 AI → 지금의 AI

분석형 AI

"입력에서 정답을 예측"

데이터로 입력과 정답 사이의 사상을 학습합니다.
예측과 분류가 분석형 AI에 속합니다.



생성형·Agentic AI

"분포에서 생성, 실행을 지휘"

학습한 분포에서 새 콘텐츠를 만들고,
도구와 루프로 일을 끝까지 수행합니다.

로직을 누가 만드는가

Software 1.0

사람이 코드를 쓴다

```
def price(x):  
    return 4*x + 2
```

로직을 사람이 직접 씁니다.

Software 2.0

데이터가 코드를 쓴다

```
w, b = fit(X, y)  
# 4와 2를 데이터가 찾는다  
def price(x, w, b):  
    return w*x + b
```

파라미터를 데이터가 찾습니다.

Software 3.0

영어로 코드를 쓴다

우리 가게 가격표를 정리해
줘

네, 정리했습니다. 품목별로
묶으면...

요청과 답변, 채팅이 곧
프로그램입니다.

Agentic

실행 통제권을 모델이 진다

요청: 시장 조사 보고서 만들어
줘



웹 조사

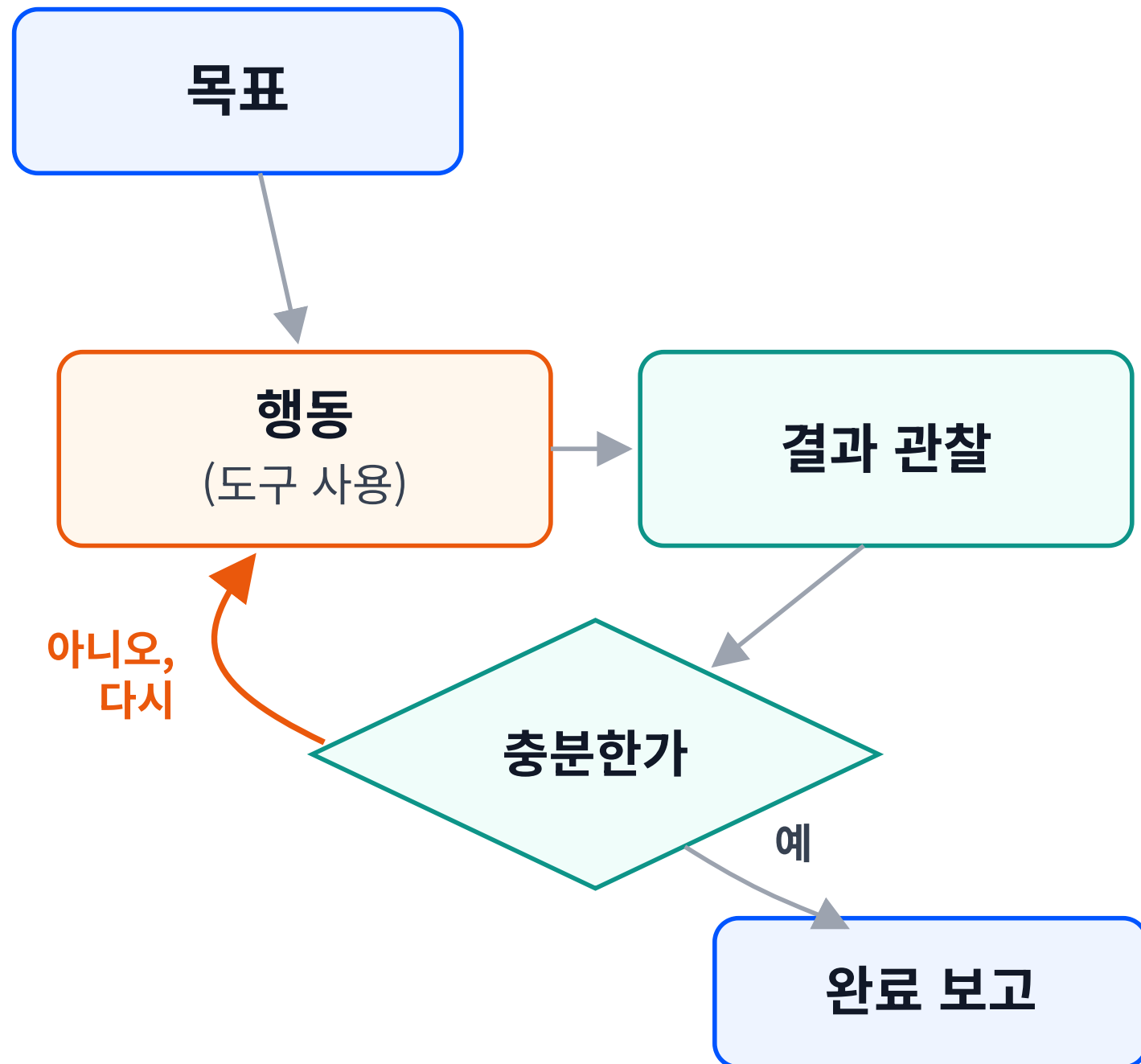


자료 정리



답변 + 보고서 파일 산출

Agentic AI는 실행의 통제권을 모델이 집니다



모델이 스스로 과정을 지휘합니다

- 도구를 골라 **호출**하고, 결과를 **관찰**하고, 충분할 때까지 **반복**합니다.
- 사람은 목표를 주고, 실행 방식의 통제권은 모델이 유지합니다.

시키면 한 번 답하는 것과 **맡기면 끝까지 해내는 것**의 차이입니다.

AI에게 일을 시키는 방식이 세 번 바뀌었고,
지금은 **부리는 단계**입니다.

실습 1 — 같은 질문, 세 곳

목표

모델마다 답의 스타일이 다름을 직접 관찰

산출물 규격

세 곳의 답 차이를 각 한 줄씩 기록

체크리스트

- 무료 웹 3곳 접속 (ChatGPT·Gemini·Claude)
- 같은 질문 1개 투입
- 스타일·길이·태도 차이 기록

도구 규칙

손 모드입니다. 직접 실행하고 직접 기록합니다.

지금부터 같은 질문 하나를 세 모델에 던지는 실습을 합니다. 판정 없이 관찰만 합니다.

"AX 전략기획과 멀티에이전트 개발 직무로 커리어를 준비 중입니다. 6개월간 무엇을 공부해야 할지 3가지로 조언해 주세요."

다섯 가문: GPT, Claude, Gemini, Grok, 오픈 모델

가문	만든 곳	대표 모델	접근 경로
GPT	OpenAI	GPT-5.6 (Sol)	웹 무료 / 구독 / API
Claude	Anthropic	Claude Fable 5 · Opus 5	웹 무료 / 구독 / API
Gemini	Google	Gemini 3.1 Pro · 3.7 Flash	웹 무료 / 구독 / API
Grok	xAI	Grok 4.6	웹 무료 / 구독 / API
오픈 모델	Meta · Alibaba · DeepSeek 외	Llama 4 · Qwen3.8 · DeepSeek V4	직접 구동 / 호스팅 API

블라인드 대전 벤치마크 상위권 (Arena Score 기준. 점수 차는 오차범위 수준, 사실상 동급)



모델명 기준: 2026-08-20, 각사 공식 문서 확인 · 벤치마크: LMArena Text Leaderboard (arena.ai, 2026-08-19 스냅샷. 막대는 1400점 기준 확대)

돈이 나가는 방식은 두 가지입니다

구독

"월 정액"

사람이 웹·앱으로 쓸 때.
많이 써도 적게 써도 요금이 같습니다.
단, 사용량 limit이 있습니다.

API 종량

"토큰 단위 과금"

서비스가 모델을 호출할 때.
입력 토큰 + 출력 토큰만큼 냅니다.

같은 가문 안에서도 등급 차이가 큼니다

모델	입력 (1M 토큰)	출력 (1M 토큰)
GPT-5.6 Sol	\$5.00	\$30.00
GPT-5.6 Luna	\$0.20	\$1.20
GPT-4o mini	\$0.15	\$0.60
Claude Opus 5	\$5.00	\$25.00
Gemini 3.7 Flash	\$0.75	\$3.75

같은 내용이라도 한국어는 영어보다 토큰을 더 씁니다.

실습 2 — 응용

쉬운 버전

자기 관심 주제로 같은 3곳 비교를 1회 더

예시: "지금 배우는 분야의 입문 자료를 3개 추천해 줘"

어려운 버전

"세 모델이 서로 다른 답을 내는 질문"을 역으로
설계하고, 왜 갈리는지 가설 1줄

예시: "AI 시대에 개발 직군은 늘어날까, 줄어들까?"

운영

모두 쉬운 버전부터 진행합니다. 먼저 끝나면 어려운
버전까지.

도구 규칙

손 모드입니다. 직접 실행하고 직접 기록합니다.

챕터 요약

- ✓ 변천의 두 질문: 로직을 누가 만드는가, 실행을 누가 지휘하는가
- ✓ 가문 지도: GPT, Claude, Gemini, Grok, 오픈 모델
- ✓ 접근 방식: 무료 웹, 구독, API 종량 (토큰 단위)

GenAI 기반 기술

LLM 텍스트 생성 원리와 생성 파라미터

원리 사슬 / 생성 파라미터 / 환각과 그라운드링

학습 목표

- LLM이 텍스트를 어떻게 생성하는지를 **입출력 구조 위에서 설명할 수 있습니다.**
- 생성 파라미터(Temperature)가 그 원리 위에서 무엇을 의미하는지 설명할 수 있습니다.
- 파라미터 차이가 만드는 실질 결과 차이를 시연하고, 응답의 근거 여부를 판정할 수 있습니다.

✓ 변천사와 모델 생태계

▶ 텍스트 생성 원리와 파라미터

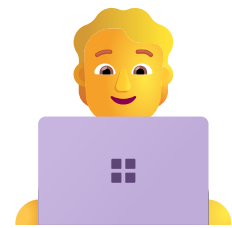
나만의 AI 어시스턴트

프롬프트의 단계적 개선

AI 코딩 에이전트 설치와 구조

멀티 LLM 통합과 첫 API 서비스

같은 질문, 다른 답



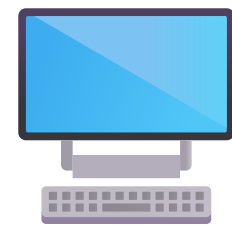
사용자

환불 규정을 세 줄로 요약해 주세요.

1. 수령 후 7일 이내에 신청하면 환불됩니다. 2. 단순 변심이면 왕복 배송비는 구매자 부담입니다. 3. 개봉하거나 훼손된 상품은 제외됩니다.

환불 규정을 세 줄로 요약해 주세요.

환불은 상품을 받은 날로부터 7일 안에 신청할 수 있습니다. 단순 변심이라면 배송비를 부담하셔야 하고, 개봉한 상품은 환불이 어렵습니다.

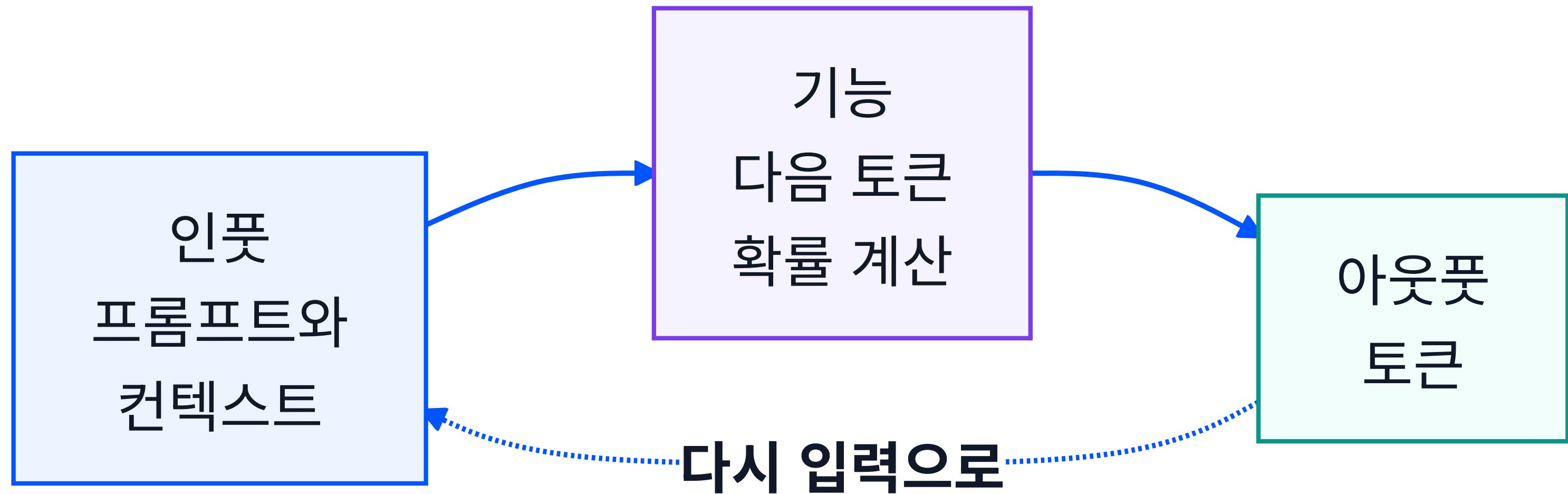


같은 모델

LLM은 다음 토큰(token)을 확률로 고릅니다.

정답을 찾아 주는 도구가 아니라, 가장 그럴듯한 다음 조각을 계산하는 도구입니다.

모델의 입출력 구조



텍스트는 조각(토큰) 단위로 처리됩니다

입력 문장

"논문을 하나 봤더니 새로운 논문이 3개 나왔다"

토큰으로 분절

논문	을	하나	봤더니	새로운
#2214	#12	#388	#7041	#916

토큰이 기본 단위인 이유

- 모델은 글자가 아니라 **조각과 번호**로 텍스트를 봅니다.
- 입력 한도도, 요금도 **같은 토큰 단위**로 계산됩니다.

번호가 아니라 좌표라야 계산이 됩니다

king : queen = prince : [?]

그냥 번호를 매기면?

왕 = 1

여왕 = 2

여왕 ÷ 2 = 왕 ... ?

여왕을 두 동강 내면 왕 두 개가 되는가

번호는 **계산이 의미를 갖지 않는다**

좌표(벡터)가 되면, 유추가 계산이 된다

king [0.2, 0.9]

queen [0.8, 0.9]

king → queen 방향

같은 방향을 prince에 적용

prince [0.2, 0.4]

? [0.8, 0.4]

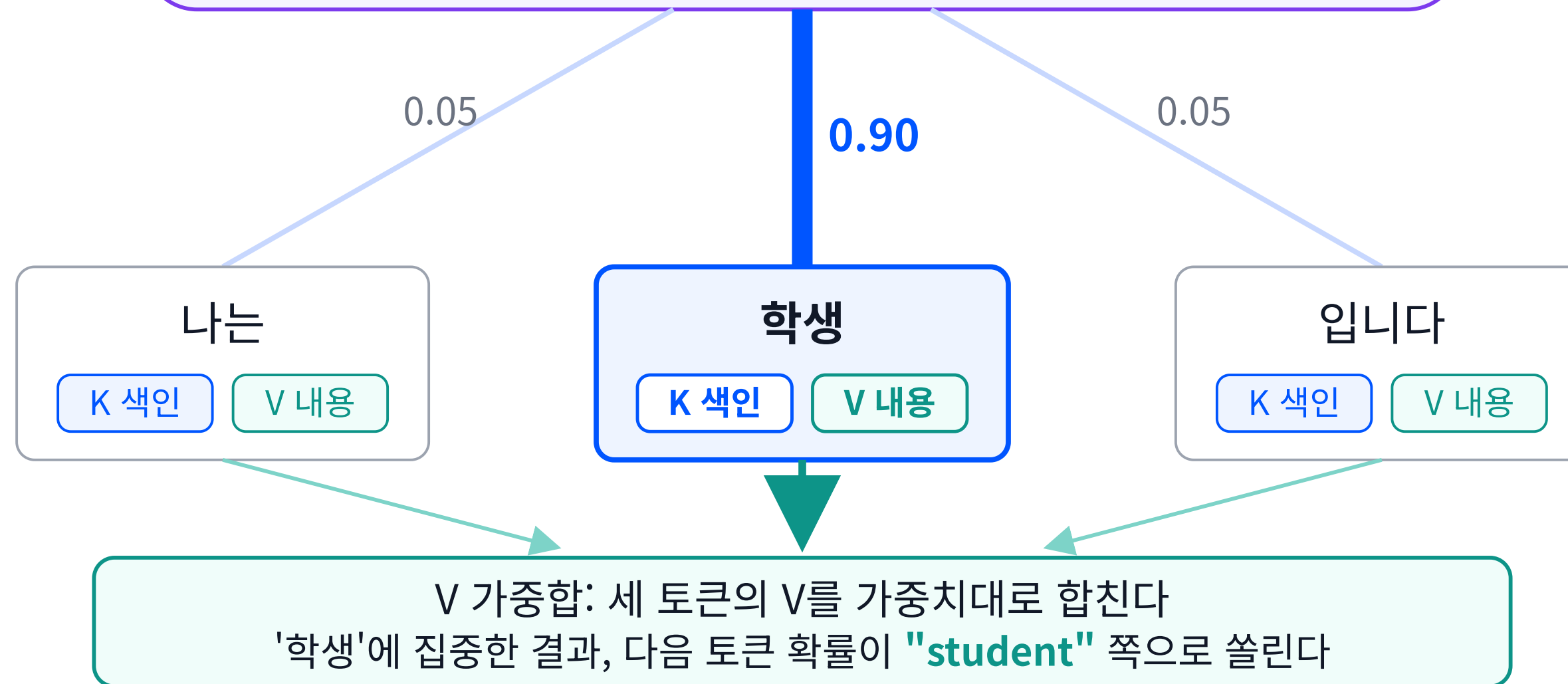
princess

토큰을 수천 차원의 좌표로 바꾸는 것이 임베딩(embedding)입니다.

다음 조각을 고르려면, 어디를 봐야 할까

"나는 학생입니다" → "<sos> I am a [?]"

Q (질문): [?]에 올 단어를 추론하기 위해 가장 유용한 정보는?



Q(질문)와 가장 비슷한 K(색인)를 찾아 V(내용)를 가중해 가져오는 것이 어텐션(attention)입니다.

모델은 다음 토큰의 확률을 계산합니다

입력

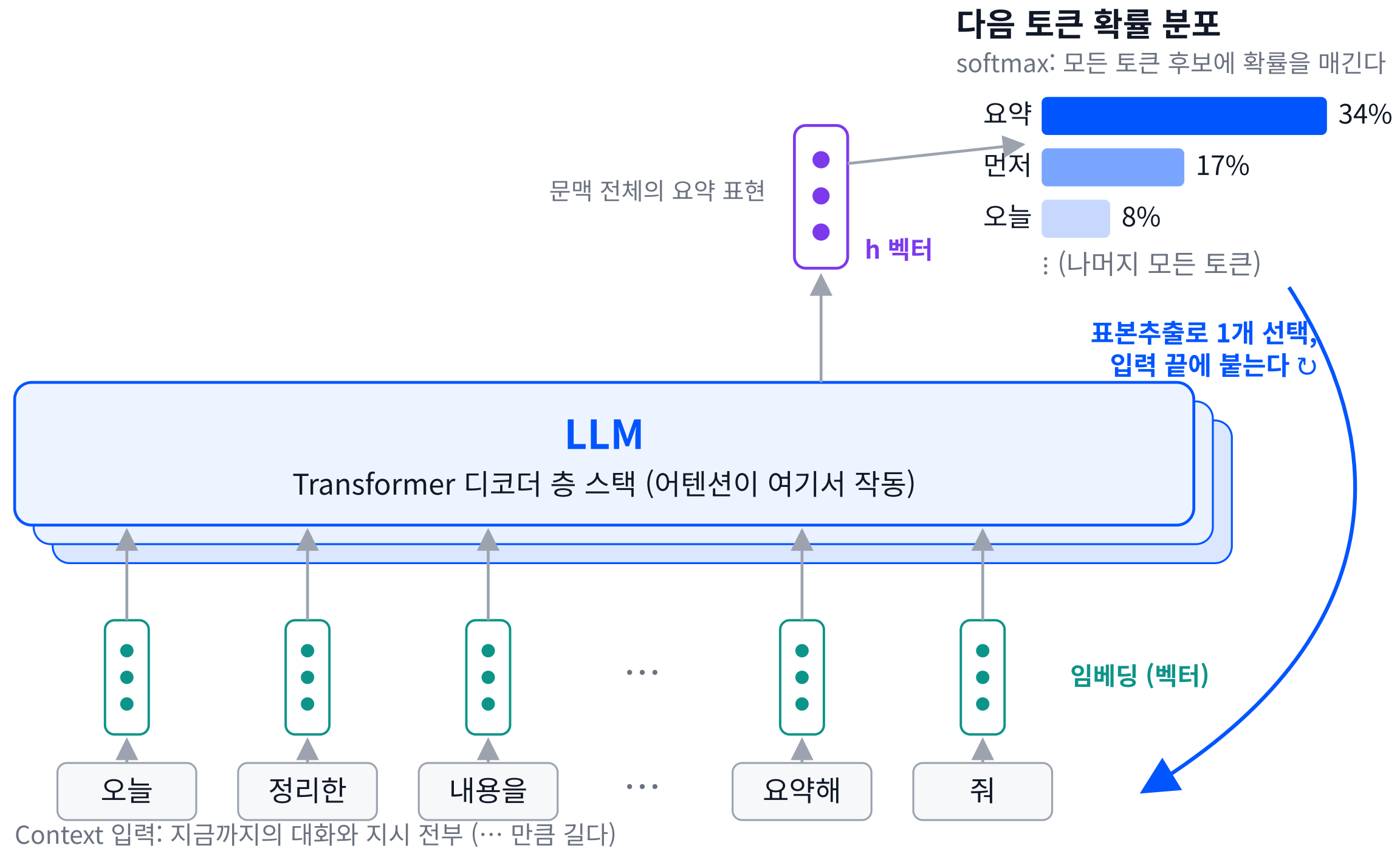
"논문을 하나 봤더니
새로운 논문이 3개 [?]"

다음 토큰 후보와 확률

후보	확률
나왔다	96.1%
생겼다	2.8%
있었다	0.9%

모델은 정답을 아는 것이 아니라, 문맥상 확률이 가장 높은 것을 계산해 낼 뿐입니다.

긴 입력이 들어가, 다음 토큰의 확률 분포가 나옵니다



구조 도해 원형: L. Voita, NLP Course For You (언어 생성 구조를 현행 디코더 단독 LLM에 맞게 재구성)

표본추출이기 때문에, 답은 매번 다를 수 있습니다

확률 계산까지
"매번 같다"

같은 입력이면 후보와 확률 분포는 같습니다.

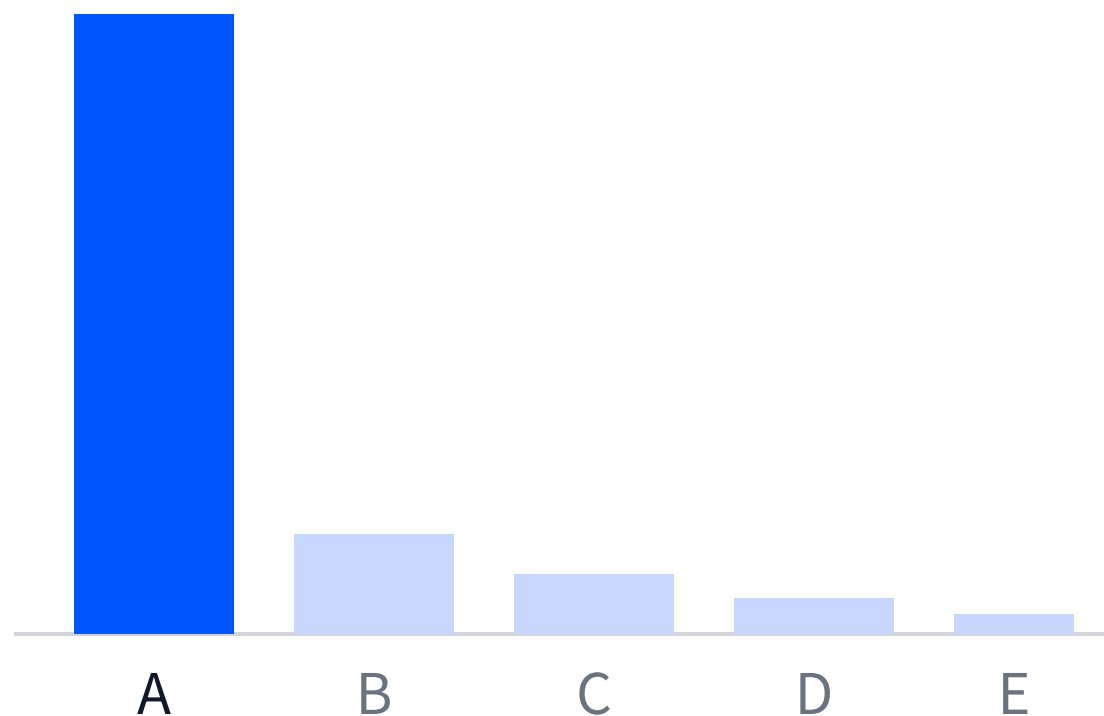


표본추출
"매번 다를 수 있다"

분포에서 하나를 뽑고,
뽑힌 토큰이 다시 입력이 됩니다.

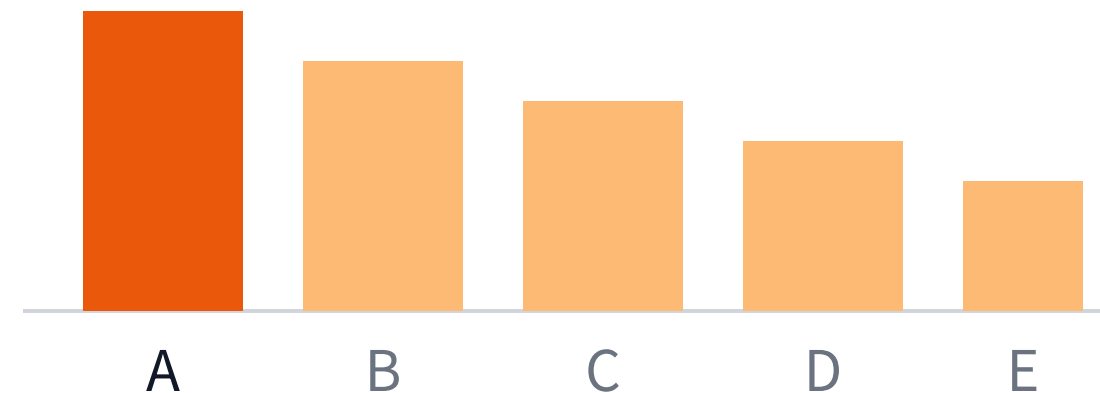
Temperature는 다음 토큰의 확률 분포를 바꿉니다

Temperature 낮음 — 분포가 뽀족하다



거의 항상 A — 결정적·일관

Temperature 높음 — 분포가 평탄하다



어느 것도 나올 수 있다 — 다양·무작위

같은 후보, 같은 모델 — Temperature는 이 분포의 모양만 바꾼다

낮으면 뽀족하게, 높으면 평탄하게 만듭니다. Temperature 0도 완전한 결정성을 보장하지는 않습니다.

후보를 잘라 내는 다이얼도 있습니다: Top-K와 Top-P

Top-K

"개수로 자른다"

확률 상위 K개 후보만 남기고
나머지는 후보에서 제외합니다.

Top-P

"누적 확률로 자른다"

확률 합이 P에 이를 때까지의
상위 후보만 남깁니다.

공식 권고는 "기본값 유지"입니다. OpenAI는 temperature와 top_p 중 하나만 조정하라 하고 ("We generally recommend altering this or top_p but not both"), Google은 기본값 유지를 강권하며, Anthropic 최신 모델은 이 값들을 아예 받지 않습니다.

용도에 맞는 Temperature

용도	Temperature	성격
정밀 작업 (데이터 추출, 요약, 코드)	낮게 (0 ~ 0.2)	일관된 답
일반 대화	중간 (0.7 안팎)	자연스러움
창작 (카피, 아이디어)	높게 (1.0 이상)	다양한 표현

다이얼은 Temperature 하나만 돌립니다. 나머지 값은 기본값 유지가 공식 권고입니다.

실습 1 — Temperature 극단 실험



지금부터 Google AI Studio에서 온도를 극단으로 바꿔 보는 실습을 합니다. 고정 프롬프트는 이것입니다:

"바다가 보이는 국내 여행지 한 곳을 골라 두 문장으로 소개해 주세요."

두 묶음의 차이를 한 줄로 기록하면 완료입니다.

실습 2 — 응용

쉬운 버전

강사 인터뷰: 던지는 질문에 답하며 Temperature 감각을 점검합니다

어려운 버전

작사 실험: 같은 요청을 Temperature 0에서 3회, 1에서 3회, 2에서 3회 반복 생성하고 변화를 관찰합니다

운영

모두 쉬운 버전부터 진행합니다. 먼저 끝나면 어려운 버전까지.

도구 규칙

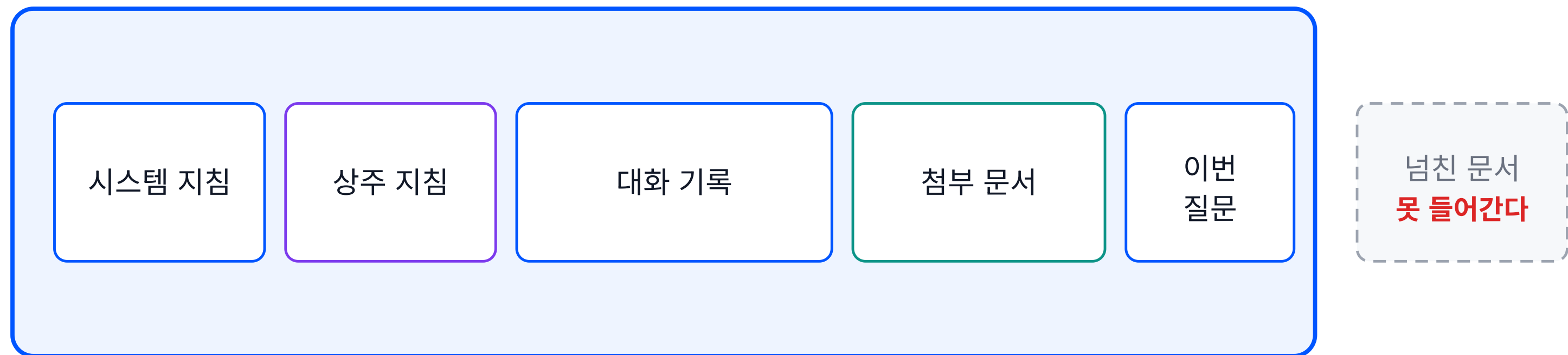
손 모드입니다. 직접 돌리고 직접 기록합니다.

작사 실험의 고정 요청 예시는 이것입니다:

"하얗게 피어난"이라는 말로 시작되는 가사를 쓰고 싶다. 한소절만 작성해줘

모델이 한 번에 볼 수 있는 양은 정해져 있습니다

컨텍스트 윈도우 — 한 번에 들 수 있는 토큰의 총량



입력과 출력이 모두 이 안에서 처리된다 — 요금도 같은 토큰 단위로 계산된다

확률로 만들기 때문에, 지어내기도 합니다



고객

가족상을 당해 급히 발권하려 합니다. 사후 할인 신청이 가능한가요?

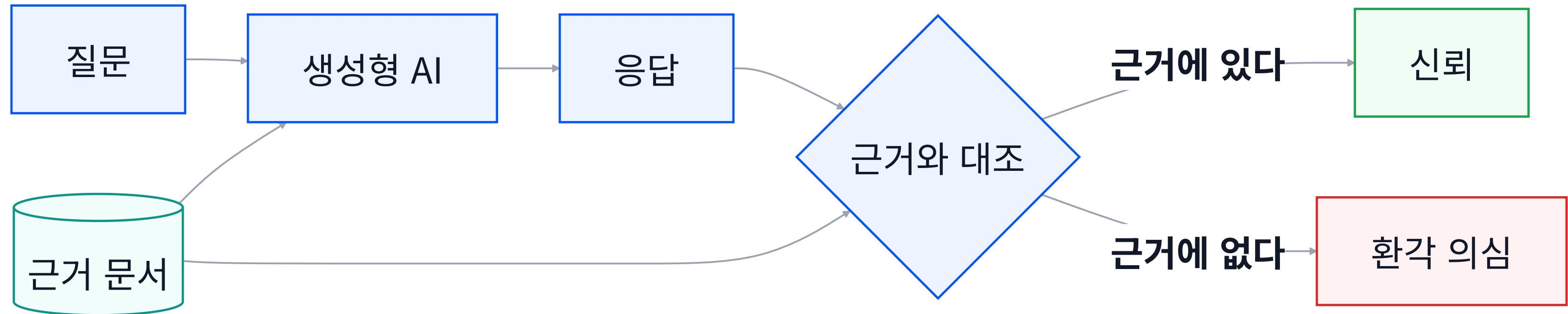
네, 가능합니다. 90일 이내에 증빙 서류를 제출하시면 차액을 환불해 드립니다.



항공사 챗봇

실제 규정에는 없는 제도였습니다. 법원은 회사에 배상을 명령했습니다.
챗봇의 안내도 회사의 공식 표시라는 판단입니다.

응답을 근거에 접지시키면 판정할 수 있습니다



그라운드링(grounding): 검증된 정보원(문서, 데이터)을 **명시적으로 참조**해 답변을 만들게 하는 것.
응답을 근거와 **대조**하면, 환각인지 아닌지를 판정할 수 있게 됩니다.

챕터 요약

- ✓ 원리 사슬: 입출력 구조 → 토큰화 → 임베딩 → 어텐션 → 확률 계산 → 표본추출
- ✓ 귀결: 같은 질문 다른 답, 그리고 환각. 대응의 이름은 그라운드링
- ✓ Temperature는 분포의 다이얼, 컨텍스트 윈도우는 입력 상한이자 요금 단위

GenAI 기반 기술

나만의 AI 어시스턴트 만들기 (Gemini Gems)

상주 지침 / 강사 시연 / 개설 실습

학습 목표

- Gemini Gems로 어시스턴트 1개를 개설하고, 지침 문서 v0(누구인가·무엇을 돕나·말투)를 작성해 첫 응답에 반영됐는지 확인할 수 있습니다.
- 상주하는 지침이 이후의 모든 응답을 바꾼다는 것을, 수정 전후의 응답 비교로 시연할 수 있습니다.

✓ 변천사와 모델 생태계

✓ 텍스트 생성 원리와 파라미터

▶ 나만의 AI 어시스턴트

프롬프트의 단계적 개선

AI 코딩 에이전트 설치와 구조

멀티 LLM 통합과 첫 API 서비스

매번 처음부터 설명하고 있습니까

1번 세션: 내 정보를 준 대화



사용자

저는 취업 면접을 앞두고 발성 연습을 하고 있습니다. 이번 주 연습 계획을 짜 주세요.

면접 대비시군요. 호흡 안정 10분과 발성 15분으로 사흘 계획을 짰습니다. 첫날은 복식 호흡부터 시작합니다.



채팅 1

2번 세션: 새 채팅



사용자

이번 주 연습 계획을 짜 주세요.

어떤 연습을 말씀하시는지 알려 주시면 계획을 짜 드리겠습니다. 운동, 악기, 공부 모두 가능합니다.

?

아니 나는 취업 면접을 앞두고 발성 연습을 하고 있는데... 저번주에 이야기 해줬잖아요??

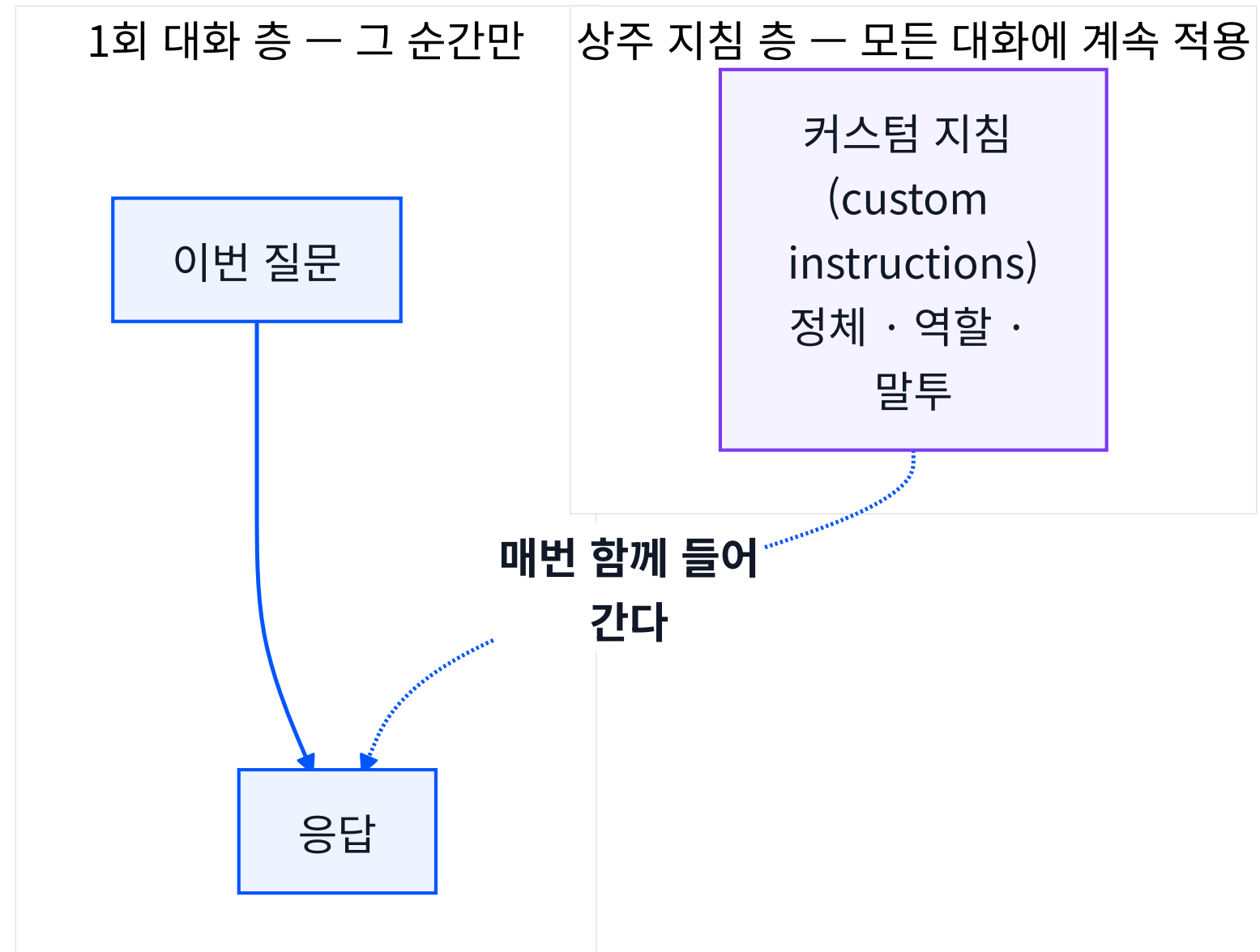


채팅 2

내 정보를 담은 지침 문서 하나를 **상주**시킵니다.

공식 이름은 커스텀 지침(custom instructions)입니다. 이 과정에서는 '영혼프롬프트'라는 별칭으로 부릅니다.

상주하는 지침 vs 1회성 대화



층이 다르면 수명이 다릅니다

- 지침 층은 **모든 대화에 계속** 적용됩니다.
- 대화 층은 **그 순간만** 유효합니다.
- 지침을 한 줄 고치면, 이후의 모든 응답이 달라집니다.

지침의 계층 구조는 뒤 강의에서 더 정밀하게 다룹니다.

같은 질문, 지침이 있을 때와 없을 때: 운동

맨 채팅

내가 팔운동을 하는게 좋을까?

팔 운동을 진행하는 것이 좋을지는 **현재 운동 목표와 전체적인 루틴 구성에 따라 달라집니다.**

팔 운동이 필요한 경우

- **특정 부위 근성장이 목표일 때:** 이두근이나 삼두근의 볼륨을 키우고 싶다면 보조 운동으로 팔 단독 운동을 추가하는 것이 효과적입니다.
- **상체 균형 및 미용적 목적:** 상체 라인을 돋보이게 하거나 옷핏을 살리고 싶을 때 직접적인 자극을 줄 수 있습니다.
- **수행 능력 향상:** 벤치프레스(삼두 관여)나 턱걸이·풀다운(이두 관여) 등 대근육 운동 시 팔 힘이 부족해 먼저 지친다면, 팔 근력을 따로 강화해 주는 것이 도움 됩니다.

팔 운동을 뒤로 미뤄도 되는 경우

- **운동 입문자 단계:** 스쿼트, 데드리프트, 벤치프레스, 바벨로우 등 대근육 중심의 복합 관절 운동만으로도 초기에는 팔이 충분히 함께 자극받아 성장합니다.
- **시간이 부족한 경우:** محدود된 시간 내에 운동해야 한다면 가슴, 등, 하체, 어깨 같은 큰 근육군에 우선순위를 두는 것이 체력 및 칼로리 소모 면에서 효율적입니다.

현재 대근육 위주의 운동을 이미 진행 중이신가요, 아니면 팔에 집중하고 싶은 특별한 목표(예:

내 어시스턴트

내가 팔운동을 하는게 좋을까?

헬스 코치 v2 · 사용자설정 Gem

형, 팔운동 추가할까 고민 중이구나? 패션 근육의 완성은 팔이긴 하지! ㅋㅋ 근데 결론부터 딱 말하자면 **지금 당장은 팔운동을 굳이 추가하지 않는 걸 추천해.** 오히려 지금은 참는 게 형의 목표 달성에 전략적으로 훨씬 유리할 거야.

그 이유를 뇌과학적 관점과 운동 생리학적 관점에서 정리해 줄게.

1. 이미 팔 근육은 최대치로 일하고 있어

형의 현재 월수금 루틴을 보면, 아이소레터럴 체스트 프레스랑 숄더 프레스를 RPT 방식으로 묵직하게 밀고 있잖아? 이때 삼두근은 이미 폭발적인 부하를 견디고 있어. 그리고 랫풀다운, 리어 풀다운, 어시스티드 풀업을 당길 때는 이두근이 강하게 개입되지. 게다가 화목토에 하는 기르보이 스윙이나 벤트 프레스 같은 케틀벨 운동은 전완근의 그립력과 팔 전체의 안정성을 엄청나게 요구해. 즉, 형의 팔은 메인 운동을 수행하는 것만으로도 지금 충분히 강한 성장 자극을 받고 있어.

2. PSMF 환경에서의 회복력 분배 (선택과 집중)

지금 형은 비만 노출 시간을 최소화하기 위해 PSMF 기반으로 극도의 칼로리 제한(VLCD)을 걸어둔 상태잖아. 이 전략은 체지방을 깎아내기엔 최고지만, 섭취 에너지가 워낙 적다 보니 신체의 전반적인 회복 용량(MRV) 자체가 평소보다 낮아져 있을 수밖에 없어. 근성장의 핵심 중 하나는 '회복'인데, 지금처럼 제한된 회복 에너지는 흉근, 등, 하체 같은 대근육을 지키고 키우는 메인 복합 관절 운동에 '몰빵'하는 게 과학적으로 훨씬 효율적이야. 팔운동(단관절

같은 질문, 지침이 있을 때와 없을 때: 발성

맨 채팅

샤우팅을 하고 싶다!

속이 답답할 때 제대로 지르는 샤우팅만큼 확실한 스트레스 해소법도 없죠. 목도 보호하면서 마음껏 소리지를 수 있는 추천 장소와 방법을 정리해 드립니다.

마음껏 소리지를 수 있는 추천 장소

- **코인 노래방:** 가성비 최고입니다. 헤비메탈이나 고음역대 곡(록, 초고음 발라드)을 예약하고 100점 신경 쓰지 말고 질러보세요.
- **달리는 차 안:** 운전 중이라면 창문을 완전히 닫고 고속도로나 도로 위에서 소리를 질러보세요. 소음 차단이 생각보다 잘 됩니다.
- **야구장/축구장:** 경기장에 가서 응원가를 목이 쉬라 불러보세요. 주변이 모두 소리를 지르고 있어서 눈치 볼 필요가 전혀 없습니다.
- **롤러코스터:** 놀이공원에서 놀이기구를 타며 비명을 지르는 것도 자연스럽게 스트레스를 날리는 좋은 방법입니다.
- **바다나 산 정상:** 탁 트인 자연을 향해 소리를 지르면 가슴 속까지 뽕 뚫리는 기분을 느낄 수 있습니다.

집에서 안전하게 지르는 팁

- **베개/이불에 얼굴 파묻기:** 집 밖으로 나갈 상황이 아니라면, 두꺼운 베개나 이불에 입을

내 어시스턴트

샤우팅을 하고 싶다!

발성 코치 v1 · 사용자설정 Gem

로큰롤의 피가 다시 끓어오르는군요! 발성 재활 전문 코치 젤민(Gemmin)입니다. 가슴 속 깊은 곳의 에너지를 폭발시키는 샤우팅은 그 자체로 엄청난 카타르시스를 줍니다.

하지만 비대칭 피열연골이라는 유니크한 커스텀 악기로 과거의 무자비한 타격감을 그대로 재현하려 한다면, 성대는 치명적인 피로도를 겪게 될 것입니다. 안전하고 건강하게, 매력적인 허스키 락 톤과 샤우팅을 만들어내는 음성과학적 타협점을 찾아야 합니다.

1. 샤우팅의 과학: 진성대(Reed)가 아닌 가성대의 예술

건강한 샤우팅과 거친 질감은 진성대(True vocal folds)를 억지로 굽어내는 것이 아닙니다.

*Sakakibara et al. (2004)*의 연구 등 다수의 음성과학 논문에 따르면, 안전한 익스트림 발성은 진성대 위쪽에 위치한 **가성대(False vocal folds)**와 **피열후두개주름(Aryepiglottic folds)**의 **수동적 진동**을 활용합니다.

현재 겪고 있는 원치 않는 갈림(Fry-like)은 비대칭 진성대가 불규칙하게 부딪히며 발생하는 '오류'에 가깝습니다. 멋진 '디스토션(Distortion)'을 원한다면, 왜곡이 발생하는 물리적 위치를 진성대가 아닌 그 윗공간으로 완전히 분리해야 합니다.

실습 1 — 어시스턴트 개설

1

Gems 만들기

Gemini에서 새 Gem을 만듭니다



2

지침 v0 작성

누구인가·무엇을 돕나·말투, 세
줄이면 됩니다



3

차이 확인

Gems가 답한 것과 맨 채팅이 답한
것의 차이를 확인합니다

지금부터 각자의 어시스턴트를 개설합니다.

예시: **"너는 내 발성 연습 코치다. 호흡과 음역 확장을 단계별로 돕는다. 전문용어는 풀어서, 한 번에 하나씩."**

※ 자기 주제로 바뀌어도 됩니다.

실습 2 — 응용과 마무리

쉬운 버전

- 역할(Role)을 부여할 것
- 목표(Goal)를 부여할 것
- 수행 방법·규칙(Guide)을 부여할 것
- 할 일(Task)은 지침이 아니라 대화에서 그때그때 줍니다

어려운 버전

- Gemini와 대화하며 커스텀 지침을 보강할 것
- 하면 안 되는 것을 추가할 것
- 본인의 선호를 추가할 것
- Gemini에게 목적을 더 설명하고, 추가할 부분 1~2개를 추천받아 지침에 추가할 것

운영

모두 쉬운 버전부터 진행합니다. 먼저 끝나면 어려운 버전까지.

예시

역할: 발성 코치 / 목표: 3개월 안에 음역 한 음 확장 /
방법: 한 번에 하나씩, 쉬운 말로, 무리한 연습은 말릴 것
(할 일은 대화에서: "이번 주 연습 계획 짜 줘")

요약

- ✓ 변천의 두 질문: 로직을 누가 만드는가, 실행을 누가 지휘하는가
- ✓ 원리 사슬: 토큰 · 임베딩 · 어텐션 · 확률 · 표본추출
- ✓ 나의 어시스턴트: 상주 지침 v0 개설 완료

GenAI 기반 기술

프롬프트의 단계적 개선: 자연어 초안, 구조화, AI 친화적 정제

개선의 사다리 / 모델 특화 / 개선 실습

학습 목표

- 프롬프트를 세 단계에 거쳐서 (자연어 초안 → 구조화 → AI 친화적 정제) 개선(Engineering)하고, 개선 전후의 출력 차이를 비교해 보일 수 있습니다.
- 제작사의 공식 문서와 예제에 따라, 프롬프트를 모델 특성에 맞추어 개선할 수 있습니다.

✓ 변천사와 모델 생태계

✓ 텍스트 생성 원리와 파라미터

✓ 나만의 AI 어시스턴트

▶ 프롬프트의 단계적 개선

AI 코딩 에이전트 설치와 구조

멀티 LLM 통합과 첫 API 서비스

프롬프트 엔지니어링의 필요성

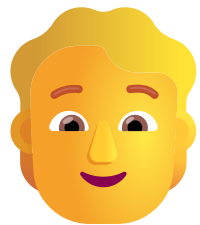
어느 커스텀 지침

"발표 준비를 도와주는 도우미입니다.
사용자가 요청하면 친절하게 잘 도와주세요."

실제로 많이 쓰는 형태의 지침입니다.

- 의도는 분명합니다.
발표 준비를 돕는 어시스턴트를 만들고 싶었습니다.
- 그런데 무엇을 돕는지(대본 정리인지 조언인지), 어떤 기준으로(길이·순서), 어떤 모양으로 답할지가 지침에 없습니다.
- 지침이 비워 둔 자리는
모델이 그때그때 임의로 채웁니다.

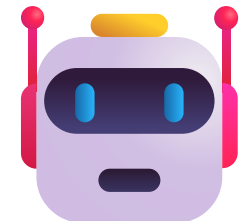
프롬프트 엔지니어링의 필요성



사용자

다음 주에 팀장님 앞에서 성과 발표가 있어요. 제가 정리한 성과 메모를 붙여 넣을게요. 이걸로 발표 대본을 잡아 주세요.

안녕하세요! 발표는 자신감이 중요합니다. 우선 인사말입니다.
"안녕하십니까, 오늘 발표를 맡은..." 성과로는 매뉴얼 제작, 응대, 교육
수강이 있으시네요. 끝으로 각오를 다지는 맺음말 예시 세 가지는...



발표 준비
어시스턴트

뭐라는 거야. 정리좀 해줘요

요청을 따르고도 결과가 난잡합니다. 기준과 형식의 빈 자리를 모델이 임의로 채웠기 때문입니다.

프롬프트는 한 번에 완성하는 것이 아니라 고쳐 가는 것입니다.

자연어 초안, 구조화, AI 친화적 정제의 세 단계를 사다리처럼 오릅니다.

모델은 명석하지만, 우리의 업무 정보를 갖고 있지 않습니다

- 모델은 **우리 회사와 우리 일을 전혀 모르는 명석한 신입사원**과 같습니다.
- 신입에게 일을 맡길 때 우리는 배경 정보와 기준과 형식을 알려 줍니다. 프롬프트도 같습니다.
- 실제로 좋은 결과를 낸 프롬프트는 평균 21단어인데, 사람들이 실제로 쓰는 프롬프트는 9단어가 안 됩니다.

자가 진단법: 배경 설명 없이 동료에게 내 프롬프트만 보여 주고 그대로 시켜 봅니다. 동료가 헛갈리면, 모델도 헛갈립니다.

개선의 사다리: 세 단계

1

자연어 초안

일단 말로 씁니다. 완벽할 필요가 없습니다. 고칠 대상을 만드는 단계입니다.



2

구조화

역할, 원칙, 금지, 출력 형식의 절로 재배열합니다. 섹션이 생기면 고칠 자리가 보입니다.



3

AI 친화적 정제

모호한 말을 지우고, 기준을 숫자로 적고, 예시를 넣습니다. 필요하다면 영어로 바꿉니다.

1단계, 자연어 초안: 일단 말로 씁니다

머릿속 생각

"보고서 정리를 시키고 싶다"

아직 프롬프트가 아닙니다. 시킬 수 없습니다.



자연어 초안 v0

"내가 이번 주에 한 일을 주면 보고서처럼 정리해 줘. 너무 길지 않게 부탁해."

말이 되는 순간, 고칠 수 있는 초안이 생겼습니다.

2단계, 구조화: 절을 나누면 고칠 자리가 보입니다

초안 v0

"내가 이번 주에 한 일을 주면 보고서처럼 정리해 줘. 너무 길지 않게 부탁해."

무엇이 부족한지 짚기 어렵습니다.



구조화 v1

역할: 주간 업무 보고 작성 도우미

원칙: 성과 중심으로 3~5개 항목을 뽑는다.

항목마다 한 문장. 너무 길지 않게 쓴다.

금지: 과장 표현, 하지 않은 일의 추측

출력 형식: 보고서처럼, 제목 1줄과 항목별 한 줄

절이 생기니 "원칙이 빈약하다"처럼 결함을 짚을 수 있습니다.

3단계, AI 친화적 정제: 모호함을 지우고 기준을 박습니다

구조화 v1

역할: 주간 업무 보고 작성 도우미

원칙: 성과 중심으로 3~5개 항목을 뽑는다.

항목마다 한 문장. 너무 길지 않게 쓴다.

금지: 과장 표현, 하지 않은 일의 추측

출력 형식: 보고서처럼, 제목 1줄과 항목별 한 줄

사람끼리는 통하지만, 강조된 두 곳은 기준이 없는
말입니다.



정제 v2

You are a weekly report assistant.

Rules: 3 to 5 items, one sentence each,
under 500 characters in total.

Think step by step:

first select, then compress, then format.

Example: "고객 응대 매뉴얼 v2 배포 완료"

기준은 숫자로, 형식은 예시로 고정했습니다.

단계를 나눠 생각하게 하는 지시(Chain of
Thought)와 영어 전환도 정제의 수단입니다.

반복 개선: 결함을 하나 찾고, 제약을 한 줄 더합니다

1회전

"회의 내용을 정리해 줘"

결함 발견: 매번 길이가
제각각입니다.

제약 한 줄 추가: "5줄 이내로."

2회전

"회의 내용을 5줄 이내로 정리해 줘"

결함 발견: 담당자와 기한이
빠집니다.

제약 한 줄 추가: "항목마다
담당자와 기한 포함."

3회전

"5줄 이내, 항목마다 담당자와 기한
포함"

결함 발견: 형식이
들쭉날쭉합니다.

제약 한 줄 추가: "항목·담당·
기한의 표로."

완벽한 첫 프롬프트는 없습니다. 결함을 하나 찾고 제약을 한 줄 더하는 반복이 프롬프트를 완성합니다.

정제에 쓰는 기본 패턴은 네 갈래입니다

관점을 고정

역할 지정
Role prompting

"You are a weekly
report assistant."

답변의 기준과 말투를
한 방향으로 고정합니다

재료를 구분

구분자 사용
Delimiters

지시는 밖에,
자료는 ``` 안에

지시문과 처리할 자료가
섞이지 않게 나눕니다

과정을 지시

행동 가이드
Guide · Chain of Thought

"SWOT 분석을 수행하라"
"Think step by step"

방법론이나 사고 과정을
순서로 지정합니다

출력을 계약

형식 지정 · 예시 제공
Format · Few-shot

"제목 1줄 + 항목별 한 줄"
예: "매뉴얼 v2 배포 완료"

모양은 규격으로 정하고,
실물은 예시로 보여 줍니다

공통 원칙: "하지 마라"의 부정형 대신, 원하는 행동을 긍정형으로 지정합니다

패턴을 많이 아는 것보다, 목적에 맞는 패턴을 골라 조합하는 것이 중요합니다

실습 1 — 내 어시스턴트 지침을 단계적으로 개선합니다

1

초안 확인

내 어시스턴트의 커스텀 지침을 엽니다



2

구조화 강화

Role, Rules, Guide, Examples, Output format의 절로 키워서 재배열합니다



3

AI 친화적 정제

모호한 말을 숫자 기준으로, 원하는 출력을 구체적인 예시로 바꿉니다. 필요하다면 영어로 바꿉니다

지금부터 각자의 어시스턴트 지침을 직접 개선합니다. 고치기 전의 지침을 복사해 두고 시작합니다. 구조화에서 멈추지 않고 **AI 친화적 정제까지** 가는 것이 이 실습의 목표입니다. 개선 전후에 같은 질문을 던져, 답이 어떻게 달라졌는지 기록합니다.

잘 만든 프롬프트의 해부

You are a weekly report assistant.

Rules:

- 3 to 5 items, one sentence each.
- Under 500 characters in total.

Guide:

- **Group the raw notes by project.**
- **Think step by step: select, compress, then format.**

Examples:

- "고객 응대 매뉴얼 v2 배포 완료"
- "신규 문의 12건 처리, 응답 4시간"

Output format:

- 제목 1줄 + 항목별 한 줄

- **역할 지정(Role)**: 답변의 관점과 기준을 고정합니다.
- **행동 가이드(Guide)**: 방법을 지시합니다.
"SWOT 분석을 수행하라"처럼 방법론을,
"단계별로 나누어 사고하라"처럼 사고 과정을 지시합니다.
후자를 Chain of Thought(CoT)라고 부릅니다.
- **예시 제공(Few-shot)**: 원하는 출력의 예시를 직접 보여 줍니다.
예시 없이 지시만 하는 방식은 Zero-shot이라고 부릅니다.
- Rules와 Output format은 기준을 숫자로, 모양을 계약으로 고정합니다.

인터넷에서 가장 흔히 만나는 이미지 프롬프트

masterpiece, best quality, ultra detailed, 8K,
golden retriever puppy, park, morning light, soft focus,
cinematic lighting, photorealistic, award winning, masterpiece,
sharp focus, depth of field, vibrant colors, bokeh,
professional photography, high resolution, masterpiece, trending

태그 나열이 통하는 모델과, 통하지 않는 모델

태그로 배운 모델

"태그가 곧 학습 라벨"

이미지에 태그를 달아 둔 자료로 학습한 계보입니다. 태그 나열이 정확하게 통했고, 그래서 이 형식이 인터넷 표준처럼 퍼졌습니다.

문장으로 배운 모델

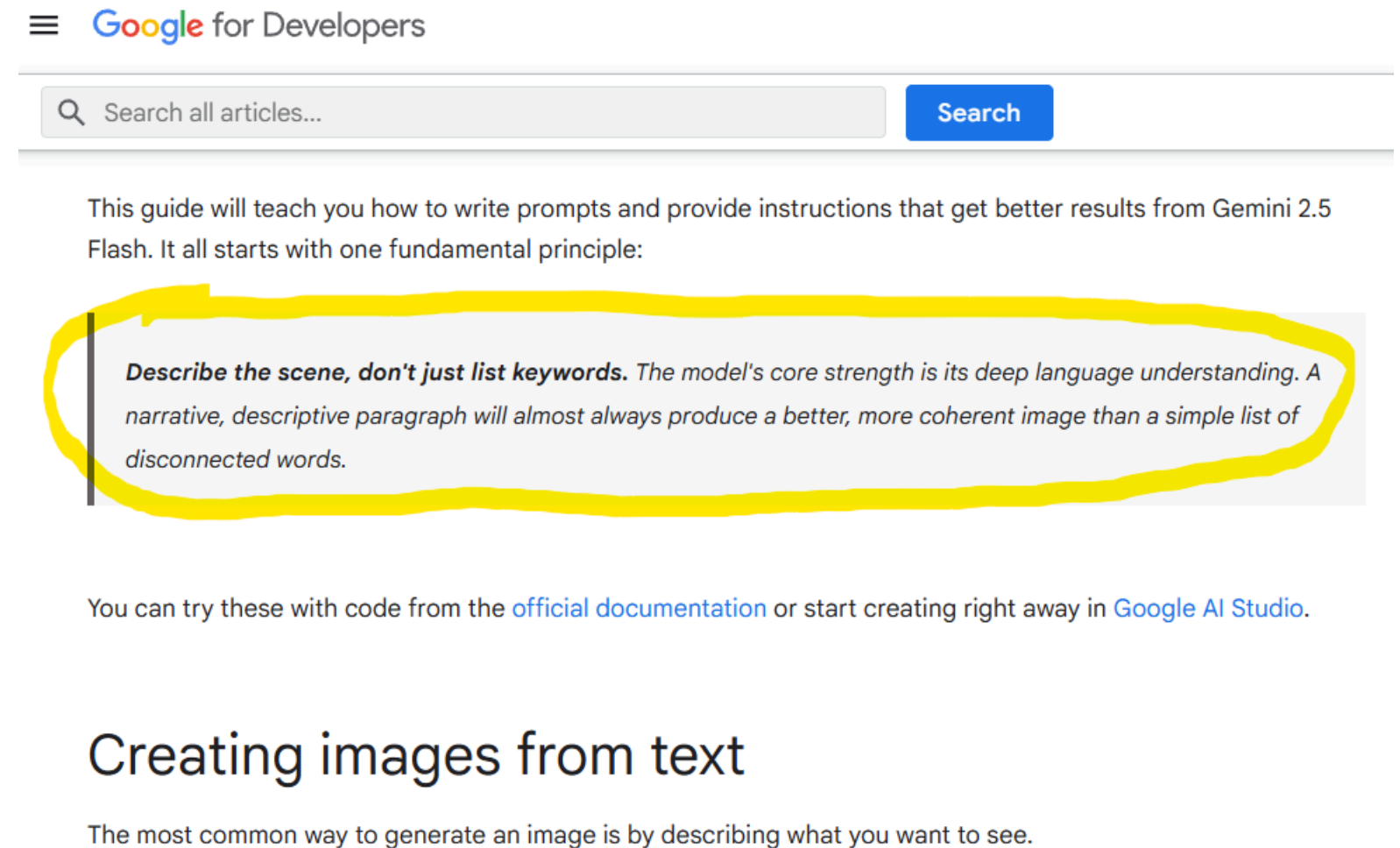
"나열 말고 서술"

언어 이해가 강점인 최신 계보입니다. 제작사 공식 가이드가 직접 말합니다. 키워드를 나열하지 말고 장면을 서술하라고요.

모델을 안 타는 표준 프롬프트는 사실상 없습니다. 프롬프트의 형식은 모델에 따라 달라지지만, 원하는 것을 구체적으로 명확하게 적는다는 원리는 어느 모델에서나 같습니다.

제작사가 밝힌 특성 ①: 장면을 서술하라

- Gemini 이미지 모델의 공식 가이드 원문입니다.
- **"Describe the scene, don't just list keywords."**
키워드를 나열하지 말고 장면을 묘사하라.
- 서사가 있는 문단이 낱말 목록보다 거의 항상 더 좋은 이미지를 만든다고 제작사가 직접 적었습니다.
- 가이드는 곧바로 장면 서술의 템플릿과 예시 프롬프트로 이어집니다.



제작사가 밝힌 특성 ②: "하지 마라"가 안 통합니다

Gemini의 문면

- Use "semantic negative prompts": Instead of saying "no cars," describe the desired scene positively: "an empty, deserted street with no signs of traffic."

"no cars"라 쓰지 말고, "텅 빈 거리"처럼
원하는 장면을 긍정문으로 서술하라.

FLUX의 문면

❗ No negative prompts: FLUX.2 does not support negative prompts. Focus on describing what you want, not what you don't want.

"FLUX.2 **does not support** negative prompts."
아예 지원하지 않는다고 명시했습니다.

같은 주제인데 제작사마다 문면이 다릅니다. 이 차이가 곧 모델 특성이고, 특성은 제작사 문서에서 확인합니다.

같은 모델 가족 안에서도, 세대가 바뀌면 가이드가 바뀝니다

항목	이전 세대 (2.5 Flash Image)	현행 세대 (3.1 Flash Image · 3 Pro Image)
세대 상태	공식 문서가 legacy로 분류, 전환 권고	공식 권장 모델
장면 서술·긍정문 전환	있음 ("no cars" 대신 "텅 빈 거리")	같은 예시 그대로 승계
작업 동사 선행	없음	신설: 만들 것을 동사로 먼저 예: "Create a poster of ..."
해상도	1024px 고정	1K 기본, 2K·4K를 코드(API 인자)로 지정 프롬프트의 "8k"는 해상도를 바꾸지 못합니다
입력 이미지 한도	공식 미공표	최대 14장 혼합 예: 3 Pro는 사물 6 + 인물 5 + 스타일 참조 3

특성을 반영하면 결과가 달라집니다

태그 나열 프롬프트



"cat, cafe, rainy evening, cozy, 8k"

장면 서술 프롬프트



"비 오는 저녁, 창가 자리에 고양이가 앉아 있는 아늑한 카페. 유리창에 빗방울이 맺혀 있다."

복잡한 장면일수록 차이가 벌어집니다



태그형 프롬프트 (전문)

1girl, solo, sitting by window, 2nd floor LP bar, rainy night, wet street, Seoul alley, neon lights, orange and red light reflections, holding warm tea cup, looking outside, melancholic eyes, glass window with water droplets, soft bokeh, cinematic lighting, 35mm film style photography, highly detailed, masterpiece, best quality, 8k resolution, trending on artstation, unreal engine 5 render, hyperrealistic

복잡한 장면일수록 차이가 벌어집니다



문장형 프롬프트 (전문)

비 내리는 늦은 밤, 네온사인이 화려하게 빛나는 서울의 오래된 골목길 2층 LP 바 창가에 한 여성이 앉아 있다. 그녀는 따뜻한 찻잔을 두 손으로 소중하게 감싸 쥐고, 유리창 밖에 쏟아지는 빗줄기와 아스팔트 젖은 바닥에 반사되는 주황색과 붉은색 네온 불빛을 아련한 눈빛으로 바라보고 있다. 창문 유리에 송글송글 맺힌 빗방울 너머로 도시의 야경이 부드럽게 보케(Bokeh)로 뭉개져 있으며, 바 실內的 은은한 주황빛 조명이 그녀의 측면 얼굴과 어깨선을 따뜻하게 감싼다. 시네마틱한 35mm 필름 스틸컷 느낌의 깊이감 있고 고독한 감성의 사진.

실습 2 — 같은 장면, 두 형식

목표

같은 장면을 태그 나열식과 장면 서술식으로 각각 생성해, 차이를 눈으로 판정합니다

도구

Gemini 웹의 이미지 생성을 사용합니다 (무료 한도는 수시로 바뀌므로 당일 안내를 따릅니다)

산출물

두 프롬프트 전문과 결과 이미지, 그리고 내 판정 한 줄

예시

태그판: "cat, cafe, rainy evening, cozy, 8k"

서술판: "비 오는 저녁, 창가 자리에 고양이가 앉아 있는 아늑한 카페. 유리창에 빗방울이 맺혀 있다."

실습 3 — 나쁜 프롬프트를 고칩니다

쉬운 버전

- 개선 대상: 나쁜프롬프트00.md
- 사다리 3단(초안 확인 → 구조화 → 정제)으로 개선하고,
단계마다 무엇을 고쳤는지 한 줄씩 기록합니다

어려운 버전

- 개선 대상: 더나쁜프롬프트00.md
- 구조는 갖췄는데 출력이 계속 어긋나는 프롬프트입니다.
세 단계 중 어디가 부실한지 진단하고 고칩니다

운영

두 파일 모두 수업 중에 배포합니다.
모두 쉬운 버전부터 진행합니다. 먼저 끝나면 어려운 버전까지.

챕터 요약

- ✓ 프롬프트는 한 번에 쓰는 것이 아니라 고쳐 가는 것입니다
- ✓ 사다리 3단: 자연어 초안, 구조화, AI 친화적 정제
- ✓ 모델 특화: 특성은 제작사 공식 문서에서 확인하고, 프롬프트를 그에 맞추습니다

GenAI 기반 기술

AI 코딩 에이전트 설치와 구조 (Claude Code)

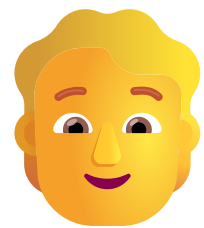
에이전트의 구조 / 환경 준비 / 이식 실습

학습 목표

- CLAUDE.md 파일이 무엇인지 알고, 직접 설정할 수 있습니다.
- 스킬·서브에이전트·훅·룰·설정이 각각 무슨 역할인지 구별해 설명할 수 있습니다.

- ✓ 변천사와 모델 생태계
- ✓ 텍스트 생성 원리와 파라미터
- ✓ 나만의 AI 어시스턴트
- ✓ 프롬프트의 단계적 개선
- ▶ **AI 코딩 에이전트 설치와 구조**
멀티 LLM 통합과 첫 API 서비스

웹 채팅으로는 안 되는 일이 있습니다



사용자

내 컴퓨터에 있는 report.md 파일 오탈자를 고쳐 주세요.

파일에 직접 접근할 수 없습니다. 파일 내용을 복사해서 붙여넣어 주시면 검토해 드리겠습니다.

파일이 40개인데요...



웹 채팅

웹 채팅의 세 가지 한계: ① 내 파일을 모릅니다. ② 반복 작업을 통째로 못 시킵니다. ③ 작업 결과가 내 컴퓨터에 남지 않습니다.

제안하는 AI에서, 일하는 AI로

코드 자동완성 세대

"다음 줄을 제안한다"

편집기 안에서 이어질 코드를 제안합니다. 실행과 판단은 전부 사람의 몫입니다.



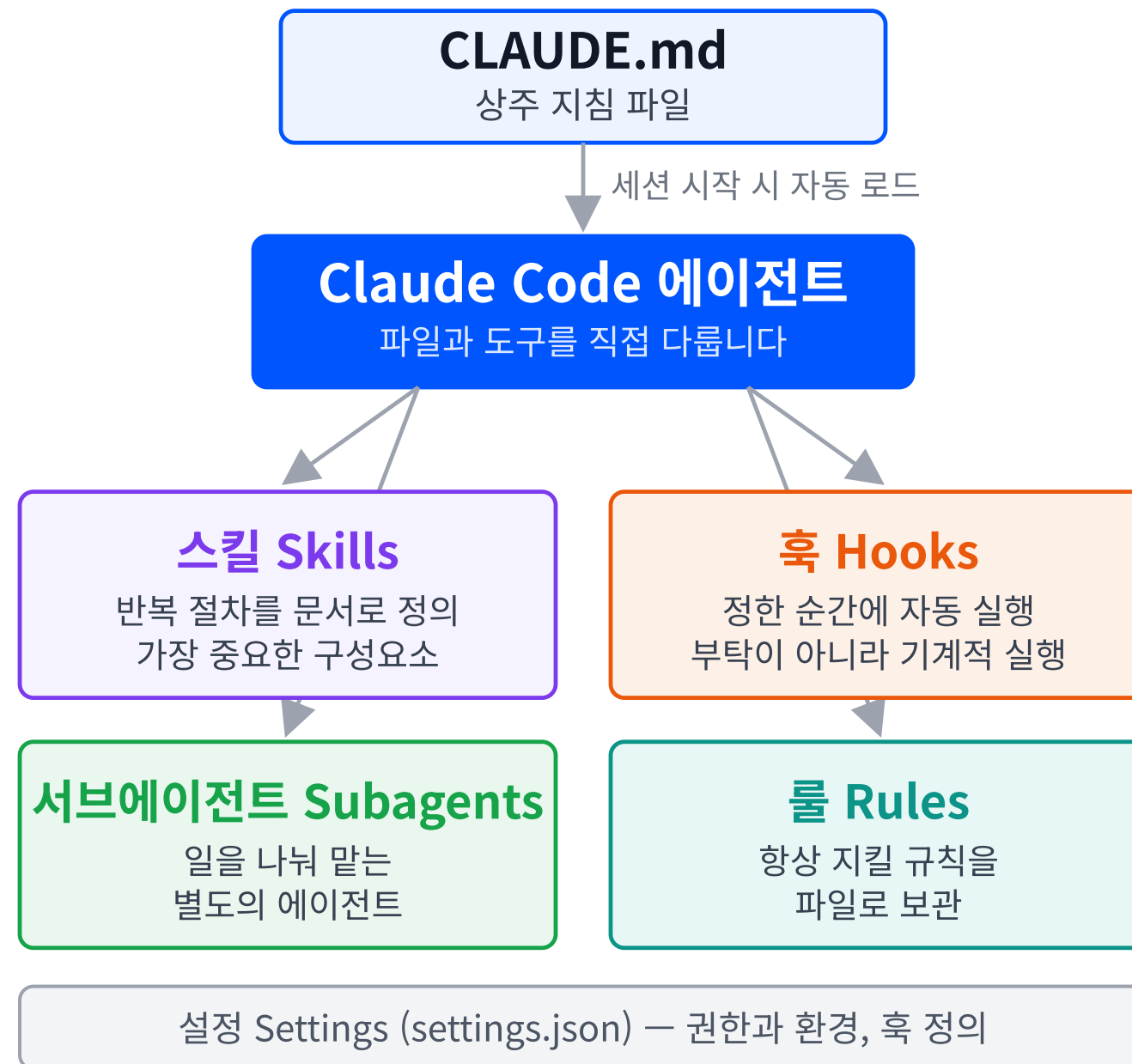
CLI 에이전트 세대

"일을 받아서 해낸다"

터미널에서 파일을 읽고 고치고, 명령을 실행하고, 결과를 확인해 다음 행동을 정합니다.

에이전트를 내 컴퓨터에 들이면, **파일과 도구를 직접 만지게** 할 수 있습니다.

Claude Code의 구조 지도



- CLAUDE.md: 세션을 시작할 때마다 자동으로 읽히는 상주 지침 파일입니다.
- **스킬(Skills)**: 반복 절차를 문서로 정의합니다. 절차의 문서화가 곧 워크플로우라서, **구성요소 중 가장 중요합니다.**
- **훅(Hooks)**: 정한 순간에 자동으로 실행됩니다. 부탁이 아니라 기계적 실행입니다.
- **서브에이전트(Subagents)**: 일을 나눠 맡는 별도의 에이전트입니다.
- **룰(Rules)**: 항상 지킬 규칙을 파일로 둡니다. 설정(Settings)은 권한과 환경입니다.

CLAUDE.md는 세 위치에 둘 수 있습니다

글로벌 (내 컴퓨터 전체)

~/.claude/

└─ **CLAUDE.md** 모든 작업에 적용

프로젝트 (이 폴더의 작업)

내 프로젝트/

└─ **CLAUDE.md** 이 프로젝트에 적용

└─ 하위 폴더/

└─ **CLAUDE.md** 그 폴더를 만질 때

가까운 지침일수록 구체적입니다

- **글로벌**에는 어디서나 지킬 내 기본 지침을,
- **프로젝트**에는 그 일의 규칙을,
- **하위 폴더**에는 그 부분에만 필요한 지침을 둡니다.
- 파일이라서 버전 관리와 재사용이 됩니다.
커스텀 지침과 달리, 지침이 서비스 안이 아니라 **내 컴퓨터의 파일로** 존재합니다.

구성요소는 전부 파일과 폴더입니다

```
~/.claude/                                글로벌 (모든 프로젝트)
├── CLAUDE.md
├── settings.json                          설정·훅(hooks) 정의
├── skills/
│   ├── security-review/
│   │   ├── SKILL.md                      절차 정의 (필수)
│   │   └── checklist.md                 스킬이 쓰는 참고 파일
├── agents/
│   └── code-reviewer.md                  서브에이전트
├── rules/
│   └── testing.md                        룰
└── 내프로젝트/.claude/                  프로젝트 전용 (같은 구조)
```

- **스킬**은 폴더 단위입니다. SKILL.md가 필수이고, 참고 문서와 스크립트를 같은 폴더에 함께 둘 수 있습니다.
- **서브에이전트**와 **룰**은 md 파일 하나씩입니다.
- 훅은 별도 파일이 아니라 settings.json 안에 정의합니다.
- 글로벌과 프로젝트, 두 그룹이 같은 구조입니다. 구성요소를 만든다는 것은 결국 **정해진 자리에 파일을 두는 일**입니다.

좋은 지침 파일의 예

- 공개된 지침 파일 하나를 같이 읽어 봅니다.
유명 연구자의 관찰을 커뮤니티가
65줄 지침 파일로 구조화한 것입니다.
- 네 가지 원칙이 뼈대입니다.
원문 전체를 두 지면에 걸쳐 실었습니다.

```
# CLAUDE.md
Behavioral guidelines to reduce common
LLM coding mistakes. Merge with
project-specific instructions as needed.
```

```
Tradeoff: These guidelines bias toward
caution over speed. For trivial tasks,
use judgment.
```

원칙	내용
가정 명시	불확실하면 추측하지 말고 물어라
최소 코드	시키지 않은 기능을 덧붙이지 마라
범위 한정	건드리라고 한 것만 건드려라
목표 주도	명령 대신 성공 기준을 줘라

지침 파일 원문 (1/2)

1. Think Before Coding

Don't assume. Don't hide confusion.
Surface tradeoffs.

Before implementing:

- State your assumptions explicitly.
If uncertain, ask.
- If multiple interpretations exist,
present them - don't pick silently.
- If a simpler approach exists, say so.
Push back when warranted.
- If something is unclear, stop.
Name what's confusing. Ask.

2. Simplicity First

Minimum code that solves the problem.
Nothing speculative.

- No features beyond what was asked.
- No abstractions for single-use code.
- No "flexibility" or "configurability"
that wasn't requested.
- No error handling for impossible
scenarios.
- If you write 200 lines and it could
be 50, rewrite it.

Ask yourself: "Would a senior engineer
say this is overcomplicated?"
If yes, simplify.

같은 원문에서, 두 원칙을 짚습니다 (1/2)

```
# CLAUDE.md
Behavioral guidelines to reduce common
LLM coding mistakes. Merge with
project-specific instructions as needed.
```

1. Think Before Coding

Before implementing:

- State your assumptions explicitly.
- If uncertain, ask.
- If multiple interpretations exist, present them - don't pick silently.
- If a simpler approach exists, say so.
- If something is unclear, stop.
- Name what's confusing. Ask.

가정 명시: 추측으로 진행하지 말고, 가정을 말하고 물으라는 지시입니다. 모호한 요청에서 엉뚱한 결과가 나오는 사고를 지침층에서 막습니다.

2. Simplicity First

- No features beyond what was asked.
- No abstractions for single-use code.
- No "flexibility" or "configurability" that wasn't requested.
- If you write 200 lines and it could be 50, rewrite it.

Ask yourself: "Would a senior engineer say this is overcomplicated?"
If yes, simplify.

최소 코드: 시키지 않은 기능을 덧붙이지 말라는 지시입니다. 코드 양이 늘수록 검토할 것도 늘어나므로, 최소가 기본값입니다.

지침 파일 원문 (2/2)

```
## 3. Surgical Changes
Touch only what you must.
Clean up only your own mess.
```

When editing existing code:

- Don't "improve" adjacent code, comments, or formatting.
- Don't refactor things that aren't broken.
- Match existing style, even if you'd do it differently.
- If you notice unrelated dead code, mention it - don't delete it.

When your changes create orphans:

- Remove imports/variables/functions that YOUR changes made unused.
- Don't remove pre-existing dead code unless asked.

The test: Every changed line should trace directly to the user's request.

```
## 4. Goal-Driven Execution
Define success criteria.
Loop until verified.
```

Transform tasks into verifiable goals:

- "Add validation" → "Write tests for invalid inputs, then make them pass"
- "Fix the bug" → "Write a test that reproduces it, then make it pass"
- "Refactor X" → "Ensure tests pass before and after"

For multi-step tasks, state a brief plan:

1. [Step] → verify: [check]
2. [Step] → verify: [check]

Strong success criteria let you loop independently. Weak criteria ("make it work") require constant clarification.

같은 원문에서, 두 원칙을 짚습니다 (2/2)

3. Surgical Changes

When editing existing code:

- Don't "improve" adjacent code, comments, or formatting.
- Match existing style, even if you'd do it differently.

The test: Every changed line should trace directly to the user's request.

범위 한정: 고치라고 한 것만 고치게 합니다. 판정 기준까지 지침에 있습니다. 바뀐 모든 줄이 요청으로 소급되어야 한다는 검사 문장입니다.

4. Goal-Driven Execution

Transform tasks into verifiable goals:

- "Fix the bug" → "Write a test that reproduces it, then make it pass"

Strong success criteria let you loop independently. Weak criteria ("make it work") require constant clarification.

목표 주도: 명령 대신 성공 기준을 주게 합니다. 기준이 검증 가능하면, 에이전트가 스스로 고치고 확인하는 반복을 돌 수 있습니다.

환경설정, 같이 합시다!

실습 1 — 기본 세팅과 학습 도우미 지침

1

설치 확인

설치 안내 문서의 확인
체크리스트를 통과합니다



2

첫 실행

실습 폴더에서 claude를
실행하고 인사합니다



3

지침 파일 배치

강사에게 제공받은 학습
도우미 CLAUDE.md
파일을 실습 폴더에 옮겨
넣습니다



4

반영 확인

학습 관련 질문을 하나
던져, 답을 바로 주지 않고
힌트로 이끄는지
확인합니다

지금부터 기본 세팅을 같이 진행합니다. 세팅은 여기까지만 합니다.
이 도구의 본격 활용은 이후의 실습들이 조금씩 쌓아 갑니다.
마지막으로 git init 한 줄로 작업 폴더의 기록을 컵니다.

반복 작업 워크플로우 구성: 학습 도우미 만들기

남은 과정 내내 여러분을 도와줄 학습 도우미를 만들어 봅시다.



실습 2 — 학습 도우미 워크플로우를 만듭니다

1단계: 직접 구성

- 강사가 나눠 주는 폴더 구조와 빈 문서 묶음을 받습니다
- 구조도를 보며 각자 빈 문서를 채워, 학습 도우미 워크플로우를 만듭니다

2단계: 점검과 패치

- 강사가 제공하는 "후속 패치 프롬프트"를 Claude Code에 넣습니다
- 원래 의도된 워크플로우와 내가 만든 것이 어떻게 다른지 확인하고 패치합니다

운영

모두 1단계부터 진행합니다. 먼저 끝나면 2단계까지.

배포물

폴더 구조 + 빈 문서 묶음, 후속 패치 프롬프트 파일.
모두 수업 중에 배포합니다.

챕터 요약

- ✓ 에이전트를 내 컴퓨터에 들이면 파일과 도구를 직접 만지게 할 수 있습니다
- ✓ CLAUDE.md는 세션마다 자동으로 읽히는 지침 파일이고, 직접 작성해 둡니다
- ✓ 구조: 스킬(가장 중요)·훅·서브에이전트·룰·설정은 각자 파일과 폴더로 존재합니다

GenAI 기반 기술

멀티 LLM 통합과 첫 API 서비스 (LiteLLM · FastAPI)

하나의 인터페이스 / 첫 API 서비스 / 통합 실습

학습 목표

- LiteLLM 단일 인터페이스로 두 개 이상의 모델을 같은 코드로 호출하고, 모델별 응답을 비교해 기록할 수 있습니다.
- FastAPI로 첫 LLM 엔드포인트를 구현해, HTTP 호출로 응답을 받아낼 수 있습니다.

- ✓ 변천사와 모델 생태계
- ✓ 텍스트 생성 원리와 파라미터
- ✓ 나만의 AI 어시스턴트
- ✓ 프롬프트의 단계적 개선
- ✓ AI 코딩 에이전트 설치와 구조
- ▶ **멀티 LLM 통합과 첫 API 서비스**

회사마다 호출법이 다릅니다

항목	GPT 계열	Claude 계열	Gemini 계열
출력 길이 인자	<code>max_completion_tokens</code>	<code>max_tokens</code>	<code>max_output_tokens</code>
시스템 프롬프트	<code>role: "developer"</code>	<code>system</code> 별도 인자	<code>system_instruction</code>
메시지 형식	<code>messages=[{role, content}]</code>	messages와 system 분리	<code>contents=[{parts}]</code>

세 회사를 쓰려면 세 벌의 코드를 관리해야 합니다. 새 모델이 나올 때마다 호출 코드를 고치는 부담이 쌓입니다.

한 회사에 묶이는 것 자체가 위험입니다

서비스 연속성

"장애가 나면 같이 멈춘다"

그 회사의 서버 장애나 일방적
규격 변경이 그대로 우리
서비스의 중단이 됩니다.

가격 주도권

"인상돼도 못 떠난다"

단가가 올라도 코드가 묶여
있으면 즉시 대응할 수
없습니다.

교체 가능한 구조

"떠날 수 있어야 자유롭다"

가장 뛰어난 모델을 고르는
일보다, 언제든지 다른 모델로
바꿀 수 있는 구조가 더
중요합니다.

모델마다 다른 호출법은 **LiteLLM** 하나로 통일하고,
내 기능에는 **FastAPI**로 주소(URL)를 붙입니다.

앞 절반이 멀티 LLM 통합, 뒤 절반이 첫 API 서비스입니다.

LiteLLM의 실체: 라이브러리 하나, 쓰는 방식 둘

Python SDK

"내 코드에 내장"

파이썬 프로그램 안에서 completion() 함수로 씁니다. 공식 문서의 표현으로는 OpenAI 클라이언트의 대체재(drop-in replacement)입니다.

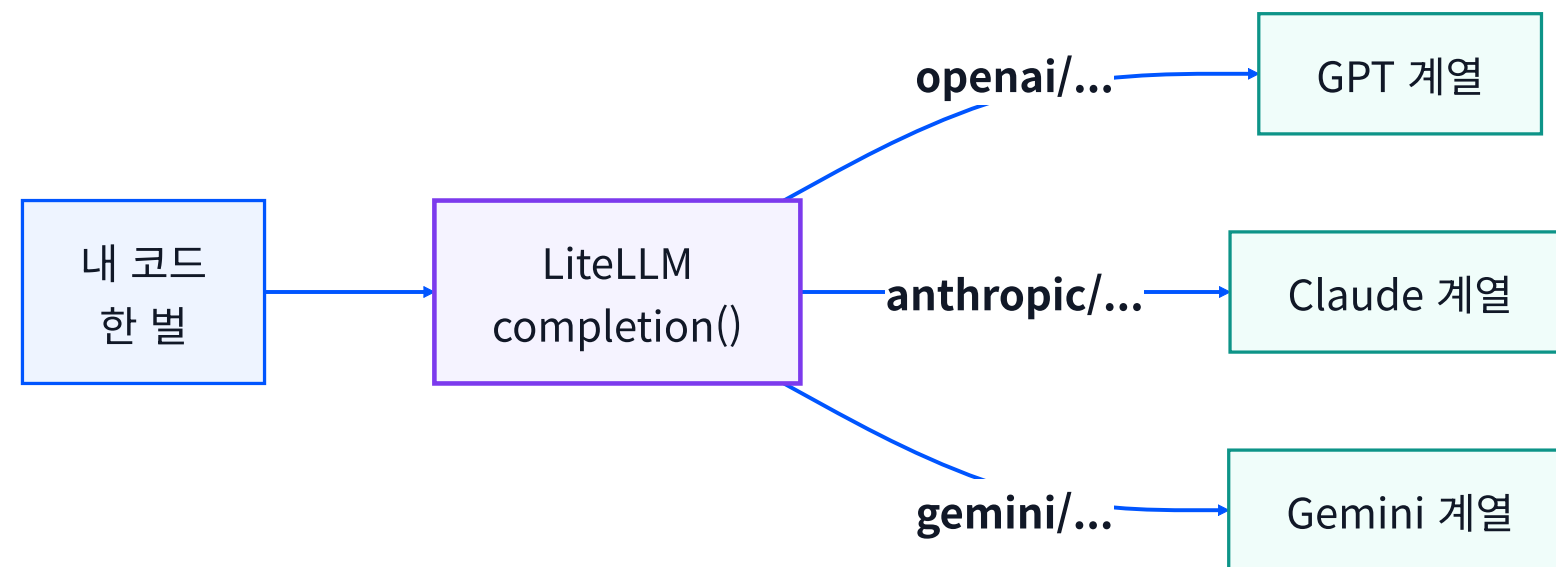
Proxy Server

"조직용 관문 서버"

회사 차원에서 모델 접근을 한곳에서 관리할 때 직접 설치해 운영하는 게이트웨이 서버입니다. 클라우드 서비스가 아니라 셀프호스팅이 기본입니다.

어느 방식이든 100개가 넘는 회사·모델을 OpenAI 형식 하나로 호출합니다. 입력 형식도, 응답 형식도 통일됩니다.

내 코드는 한 벌, 모델은 문자열로 고릅니다



- LiteLLM은 여러 회사의 호출법을 **completion() 함수 하나로** 감쌉니다.
- 모델은 "**회사/모델명**" 문자열로 고릅니다. 코드는 그대로 두고 문자열만 바꿉니다.
- 교체 가능한 구조가 이 한 줄에서 시작됩니다.

호출 한 번의 전체 모습

```
from litellm import completion

response = completion(
    model="openai/gpt-5.6-luna",
    messages=[{"role": "user",
                "content": "안녕하세요"}],
)

print(response.choices[0]
        .message.content)
```

임포트 하나, 함수 하나입니다

- **model**: 어느 모델을 부를지 정하는 문자열입니다.
- **messages**: 대화 내용입니다. role과 content의 목록으로 보냅니다.
- 응답은 response 안에 들어 있고, 답 문장을 꺼내 씁니다.

같은 코드, 모델 문자열만 바꿉니다

```
from litellm import completion

response = completion(
    model="gemini/gemini-3.1-flash",
    messages=[{"role": "user",
               "content": "안녕하세요"}],
)

print(response.choices[0]
      .message.content)
```

바뀐 곳은 한 줄뿐입니다

- **"회사/모델명"** 규격입니다. 빗금 앞이 회사, 빗금 뒤가 모델입니다.
- openai/... , anthropic/... , gemini/... 처럼 회사 접두어를 바꾸면 다른 회사의 모델을 같은 코드로 부릅니다.
- 예: openai/gpt-5.6-luna
anthropic/claude-opus-5
gemini/gemini-3.1-flash

답을 꺼내는 자리도 항상 같습니다

```
from litellm import completion

response = completion(
    model="openai/gpt-5.6-luna",
    messages=[{"role": "user",
               "content": "안녕하세요"}],
)

print(response.choices[0]
      .message.content)
```

어느 회사 모델이든 꺼내는 방법이 같습니다

- **response.choices[0].message.content**
답 문장이 항상 이 자리에 있습니다.
- 회사가 달라도 응답의 모양이 통일되어 있어서,
뒤에 붙는 내 코드는 바뀌지 않습니다.

파라미터는 이렇게 조정합니다

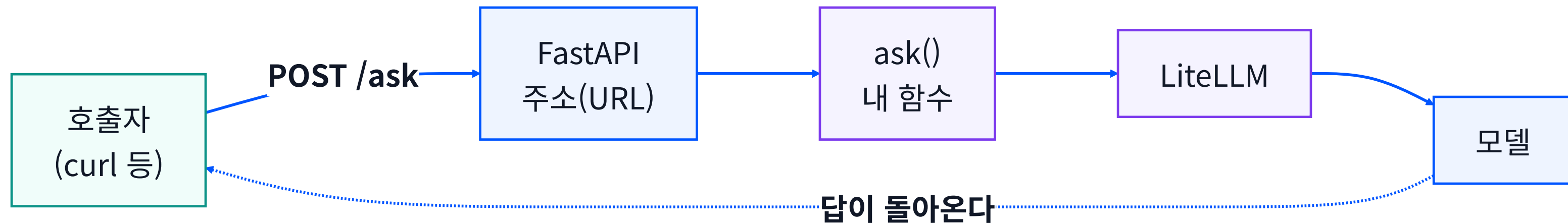
```
response = completion(  
    model="openai/gpt-4o-mini",  
    messages=[{"role": "user",  
                "content": "..."}],  
    temperature=1.4,  
)
```

웹 화면의 설정값이 함수 인자가 됩니다

- **temperature**를 코드에서 직접 지정합니다.
값을 바꿔 가며 응답 변화를
코드로 실험할 수 있습니다.
- 시연 모델은 temperature 조정을 허용하는 모델
(gpt-4o-mini)입니다.
- Top-P 등 다른 샘플링 파라미터도 같은 자리에
들어갑니다.
- 공식 사이트를 참고하여 모델별 상세 튜닝과
비용 등을 확인하는 습관을 기릅니다.

내 기능이 주소를 가지면, 남이 부를 수 있습니다

호출자가 주소로 질문을 보내면, 내 함수가 모델을 부르고, 답이 호출자에게 돌아갑니다.



FastAPI 최소 골격

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class AskRequest(BaseModel):
    question: str

@app.post("/ask")
def ask_endpoint(req: AskRequest):
    answer = ask(req.question)
    return {"answer": answer}
```

앱 하나, 주소 하나

- `app = FastAPI()` 가 서비스의 몸체입니다.
- `@app.post("/ask")` 가 주소를 붙입니다.
/ask로 들어온 요청이 이 함수로 옵니다.
- 요청의 모양은 AskRequest가 계약합니다.
question이라는 글자 하나를 받습니다.

※ 주소를 부르는 방식(HTTP 메서드)에는 종류가 있습니다. **GET**은 조회(주소만으로 요청), **POST**는 본문을 실어 보내는 요청입니다. 질문 문장을 실어 보내야 하므로 여기서는 POST를 씁니다.

코드 런 ①: ask 함수

```
def ask(  
    question: str,  
    model: str = "openai/gpt-5.6-luna",  
) -> str:  
    """질문을 받아 LLM 답변을  
    돌려준다."""  
    response = completion(  
        model=model,  
        messages=[{"role": "user",  
                    "content": question}],  
    )  
    return (response.choices[0]  
            .message.content)
```

함수의 계약부터 씁니다

- **타입힌트**: 무엇을 받아 무엇을 주는지 함수 머리에 적습니다.
- **docstring**: 함수가 하는 일을 한 줄로 적습니다.
- 모델명을 인자로 빼 두면, 부르는 쪽에서 모델을 고를 수 있습니다.

코드 런 ②: 주소 붙이기

```
app = FastAPI()

class AskRequest(BaseModel):
    question: str

@app.post("/ask")
def ask_endpoint(req: AskRequest):
    answer = ask(req.question)
    return {"answer": answer}
```

만든 함수에 주소를 연결합니다

- **/ask 주소**로 들어온 요청이
- **ask 함수**를 호출하고, 결과를 돌려줍니다.
- 함수와 주소가 분리되어 있어서, 함수는 나중에 다른 곳에서도 재사용할 수 있습니다.

코드 런 ③: 서버를 켭니다

```
$ fastapi dev day02_module.py

FastAPI   Starting development server
          Serving at:
          http://127.0.0.1:8000
```

실행 명령은 한 줄입니다

- `fastapi dev 파일명` 이 개발 서버를 켭니다.
공식 문서가 안내하는 표준 명령입니다.
- 켜져 있는 동안 주소가 살아 있습니다.
끄기 전까지 누구든 부를 수 있습니다.

코드 런 ④: 호출이 성립하면 완성입니다

```
$ curl -X POST
  http://127.0.0.1:8000/ask
  -H "Content-Type:
    application/json"
  --data-binary "@body.json"
```

```
{"answer": "FastAPI는 파이썬으로
API를 빠르게 만드는
웹 프레임워크입니다."}
```

호출 한 줄이 완성의 기준입니다

- curl은 터미널에서 주소를 부르는 도구입니다. 브라우저 없이 서비스를 검증합니다.
- 질문은 body.json 파일에 담아 보냅니다. 파일 내용은 JSON 한 줄입니다:

```
{"question": "FastAPI가 무엇인지 한 문장으로 답해 주세요."}
```

- 지면의 줄바꿈은 보기용입니다. 실제로는 한 줄로 칩니다. 한 줄 전체는 배포되는 실습 파일에 있습니다.

※ 한글 질문은 반드시 파일로 보냅니다. 윈도우 터미널에서 한글을 명령줄에 직접 쓰면 깨진 채 전송됩니다. 파일은 UTF-8로 저장합니다.

모듈을 모아 하나의 서비스로

강의 기간동안 계속 만들게 되는 모듈들을 모아, 하나의 서비스를 만들어 볼 겁니다.

1

함수 1개

타입힌트와 docstring을 갖춘 함수



2

라우터 1개

POST 주소 하나, JSON을 받고
JSON을 돌려줍니다



3

호출 성립

curl 한 줄로 동작을 확인합니다

서비스가 멈추는 이유

원인	이름
HTTP 429	할당량 초과 (Rate Limit)
HTTP 503	서버 장애 (Server Outage)
Timeout	응답 지연

공급자의 이슈는 자연재해와 같고, 우리는 대비해야 합니다.



Fallback을 준비합시다

- 모델 교체가 문자열 하나이므로, **막히면 다른 모델로 넘기는 전환**이 가능해집니다.
- LiteLLM은 이 전환을 **Router**라는 이름의 기능으로 제공합니다. 여러 모델에 요청을 나누고, 실패하면 예비 모델로 넘깁니다.

작업 특성에 따라 모델을 고르는 눈도 여기서 출발합니다. 빠르고 싼 모델과 깊고 비싼 모델을 한 코드에서 골라 쓸 수 있기 때문입니다.

Fallback 주의: 예비 모델이 같은 인자를 받는지 확인합니다

```
litellm.BadRequestError:  
OpenAIException  
Unsupported value: 'temperature'  
does not support 1.4 with this  
model. Only the default (1)  
value is supported.
```

실제 에러 메시지입니다

- 모델명 문자열만 바꾸는 수준의 Fallback은 여기서 다시 실패할 수 있습니다. 같은 인자를 어떤 모델은 받고, 어떤 모델은 **거부합니다**.
- 이 모델은 temperature를 기본값으로 고정해 두었습니다. 예비 모델의 파라미터 수용 여부까지 확인해야 Fallback이 성립합니다.
- 파라미터 수용 여부도 모델 특성입니다. 특성은 추측하지 말고, 문서와 실험으로 확인합니다.

실습 1 — 첫 모듈을 완성합니다



지금부터 스켈레톤의 빈칸을 채워 첫 모듈을 완성하는 실습을 합니다.
완성 기준은 세 가지입니다: **함수 1개, 라우터 1개, 호출 성립.**

챕터 요약

- ✓ 모델마다 다른 호출법은 LiteLLM의 completion() 하나로 통일합니다
- ✓ 모델은 "회사/모델명" 문자열로 고릅니다. 문자열 교체가 곧 모델 교체입니다
- ✓ FastAPI는 내 함수에 주소(URL)를 붙여, HTTP 호출로 쓸 수 있게 만듭니다
- ✓ 서비스 중단 세 원인은 할당량 초과, 서버 장애, 응답 지연이고,
대비는 다른 모델로 넘어갈 수 있는 구조입니다

